Manuel d'installation et de configuration - pili-pos-android v1.0.0

Plateforme : Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google)

Ce manuel s'adresse au technicien installateur (= distributeur, nuagemagique.dev). Il décrit la mise en place complète d'une caisse pili-pos-android chez un client HoReCa. Le terminal est un appareil Android verrouillé : ni console, ni clavier de maintenance, ni accès à distance. L'ordre est donc fixe : **provisionner le terminal avant tout compte Google** (§3), importer le lot de configuration signé (§3.4), contrôler la caisse (§4), puis ne plus toucher à l'appareil autrement que par l'application (§6, §9).

Le moteur fiscal est le même code source que sous Linux : `pos.json` (§4.2) est le même, seule sa livraison change. Le verrouillage est décrit dans `PLATEFORME_ANDROID_FR.md` §3, le stockage dans §5.

1. Matériel requis

Élément	Modèle / spécifications	Qté
Terminal POS	Sunmi D3 Pro avec services Google, Android 13, écran 1920 x 1080 en paysage (`PLATEFORME_ANDROID_FR.md` §1). Faits du vrai terminal : À effectuer (EV-15 : fiche mesurée d'un vrai D3 Pro)	1
Boîtier FDM	FDM module2 (PRACSYS, certifié SCE 2.0) ou autre FDM certifié SCE 2.0	1
Imprimante thermique	Epson TM-m30III, raccordée au LAN (le POS imprime en TCP brut, port 9100)	1
Tiroir-caisse	Standard, connexion RJ11 vers l'imprimante	1
Câbles Ethernet	RJ45 Cat-5e ou supérieur : FDM et imprimante (et terminal si son modèle offre une liaison filaire, sinon Wi-Fi)	2 ou 3
Switch / box internet	4 ports minimum	1
Clé USB de mise en service	Toute taille, FAT32 ou exFAT : elle porte le lot signé (§2)	1
Carte microSD (recommandée)	microSDXC de qualité, exFAT, 64 ou 128 Go : exports vers le SPF et mises à jour, jamais la base (§8)	1
Lecteur de badges (option)	Lecteur de cartes NFC avec câble USB qui se comporte comme un clavier (il tape le numéro de la carte puis Entrée) ; à brancher à l'adaptateur USB du Sunmi D3 Pro. **L'unité principale du D3 Pro n'a pas de NFC** ; son écran client optionnel est annoncé avec NFC, à confirmer sur place (§4.4 bis)	0 ou 1
Poste du technicien	Ordinateur avec `adb`, `shasum` et `minisign` (§3.2) ; onduleur de 600 VA recommandé pour la caisse	1

Ni puce TPM ni réglage BIOS (`SECURITE_BD_FR.md` §3.6). **Le terminal ne doit jamais être relié à un compte**

Google (§3.1).

2. Préparation du lot de mise en service et de la clé USB

2.1 Le lot de configuration signé

La configuration (identité fiscale, FDM, imprimante, secrets) ne se saisit pas à l'écran : elle arrive dans un **lot**, dossier préparé sur le poste du technicien (format : `docs/android/APP.md` §3) :

Fichier	Rôle
`pos.json`	La configuration de la caisse (§4.2)
fichiers que `pos.json` désigne	Jeton du FDM, `ca.crt`, `client.crt`, `client.key` de module2, jeton d'export… ; chemins relatifs au dossier du lot
`seed.sql` (facultatif)	Premiers comptes et catalogue, ou graine du jeu de test du SPF (§4.4) ; appliqué à une base qui n'existe pas encore
`MANIFEST.json`	Liste triée des autres fichiers avec taille et SHA-256
`MANIFEST.json.minisig`	Signature minisign préhachée (option par défaut de `minisign -S`) du manifeste

La clé de signature du lot appartient au producteur et ne quitte pas son poste. L'APK de production n'accepte que les lots signés par la clé dont la partie publique lui a été fournie à la compilation (sans clé : « Cette version ne contient pas de clé de vérification »). Le script `scripts/android/make_dev_bundle.sh` écrit un lot de **développement** (simulateurs, clé de développement, `--seed` pour la graine du jeu de test) : il sert aux essais sur l'émulateur avec des versions de débogage, jamais à une livraison. Un lot de livraison, machine de test comprise, a le même format et se signe avec la clé de production.

2.2 Écrire le lot sur la clé USB

Copier le dossier du lot à la racine de la clé (un lot par clé) et en vérifier la signature depuis le poste avant de partir :

```
minisign -V -p <clé publique de production> -m lot/MANIFEST.json -x lot/MANIFEST.json.minisig
```

La clé contient le jeton et la clé privée mTLS du FDM : l'effacer après l'installation, ne jamais la laisser chez le client.

3. Installation sur le terminal POS

3.1 Contrôles avant de commencer

- Le terminal est **neuf ou réinitialisé et ne porte aucun compte** (Réglages, Comptes : vide) : c'est la seule condition où le rôle de propriétaire de l'appareil peut être attribué (`PLATEFORME_ANDROID_FR.md` §3.1).
- Dans l'assistant de configuration d'Android, **ne jamais se connecter à un compte Google** : « Ignorer » à chaque proposition de compte, de sauvegarde ou de restauration.
- L'APK est celui de la version déclarée (artefact public `declared-android`). Vérifier ses empreintes avant tout :

  ```
  shasum -a 1 pili-pos-android-1.0.0.apk       # SHA-1, égal à EV-01
  shasum -a 256 pili-pos-android-1.0.0.apk     # SHA-256, égal à EV-01
  ```

  Valeurs : À effectuer (EV-01 : SHA-1 et SHA-256 de l'APK signé `pili-pos-android 1.0.0`). Une version de développement (suffixe `-dev`) n'est jamais installée chez un client.

3.2 Provisionnement au banc du producteur (seule voie de la version 1.0.0)

Le terminal est préparé chez le producteur avant la livraison. Le technicien le relie à son poste par USB (débogage USB activé) et exécute :

```
adb install -r pili-pos-android-1.0.0.apk
adb shell dpm set-device-owner dev.nuagemagique.pilipos/.provisioning.PiliAdminReceiver
adb shell dpm list-owners
```

1. **Lire le texte de la réponse**, pas le code de sortie (0 même en cas de refus) : seul « Success: Device owner set to package … » est un succès. Un refus (compte présent, plusieurs utilisateurs) se traite par la réinitialisation d'un terminal **neuf ou retiré du service** puis un nouveau provisionnement au banc, jamais en forçant.
2. `dpm list-owners` doit nommer l'application `DeviceOwner`.
3. Au lancement, l'application pose le mode kiosque et les restrictions (`PLATEFORME_ANDROID_FR.md` §3.2 à §3.4) et affiche « Mise en service de la caisse » (§3.4).
4. Tout message d'Android ou de Play Protect pendant l'installation (application non reconnue, analyse proposée) est **noté avec une capture, sans se connecter pour le faire disparaître**, au rapport d'essai. Sur le profil émulé avec services Google, aucun message de Play Protect n'est apparu à l'installation ni à la mise à jour (EV-13, `evidence/gms/EV13_2026-10-15.md`) ; sur un vrai terminal : À effectuer (EV-15 : Play Protect sur le D3 Pro réel).
5. Faire d'abord tout ce qui demande adb, et **joindre le réseau Wi-Fi du site avant l'import du lot** (§4.3) : au banc, soit dans l'assistant de configuration d'Android, en passant l'étape du compte Google, soit par adb, par exemple `adb shell cmd wifi connect-network <SSID> wpa2 <mot de passe>`. Le terminal garde ce réseau. À la fin du provisionnement d'une version de production, l'application coupe le débogage USB (`PLATEFORME_ANDROID_FR.md` §3.5). Un terminal mal provisionné se reprend alors au banc (après réinitialisation d'un terminal neuf ou retiré du service) ou par une version déclarée plus récente (§6).

3.3 Provisionnement sur le terrain, par code QR (cible, non disponible en 1.0.0)

Comme construit en 1.0.0, ce parcours ne peut pas être suivi. Le composant d'administration de l'application est déclaré (réception `DEVICE_ADMIN_ENABLED`), mais aucune activité ne répond aux deux actions que l'assistant de configuration d'Android 12 et suivants exige pour un terminal entièrement géré (`android.app.action.GET_PROVISIONING_MODE` et `android.app.action.ADMIN_POLICY_COMPLIANCE`) : l'assistant ne peut pas l'achever sur l'API 33. La version 1.0.0 se provisionne donc **uniquement au banc** (§3.2). Ce qui suit décrit la cible, pour mémoire ; il ne se suit pas sur une caisse livrée. Manquent : les deux activités de provisionnement et l'essai sur un vrai D3 Pro : À effectuer (EV-15 : activités de provisionnement et essai du parcours QR sur un vrai D3 Pro). Un terminal à reprendre se réinitialise donc chez le producteur, puis se provisionne au banc.

Cible. Réservé à un terminal **réinitialisé et sans donnée à conserver** (neuf, de remplacement, ou retiré du service après export des données). La réinitialisation d'usine efface la base fiscale : elle n'est jamais faite sur une caisse en service (§9).

1. À l'écran d'accueil de l'assistant de configuration du terminal réinitialisé, **toucher six fois au même endroit** jusqu'à l'ouverture du lecteur de code QR. Ne saisir aucun compte Google.
2. Scanner un code QR dont le contenu est ce JSON : composant d'administration, URL de l'APK signé et son SHA-256 (base64 URL, sans remplissage). Sans câble, ajouter les clés Android de Wi-Fi (`PROVISIONING_WIFI_SSID`, `PROVISIONING_WIFI_PASSWORD`).

```
{
  "android.app.extra.PROVISIONING_DEVICE_ADMIN_COMPONENT_NAME":
    "dev.nuagemagique.pilipos/.provisioning.PiliAdminReceiver",
  "android.app.extra.PROVISIONING_DEVICE_ADMIN_PACKAGE_DOWNLOAD_LOCATION":
    "<URL HTTPS de l'APK signé de la version déclarée>",
  "android.app.extra.PROVISIONING_DEVICE_ADMIN_PACKAGE_CHECKSUM":
```

```
    "<SHA-256 de l'APK, base64 URL sans remplissage>"
}
```

Empreinte du QR : `openssl dgst -sha256 -binary pili-pos-android-1.0.0.apk | openssl base64 | tr '+/' '-_' | tr -d '='`.

3. Le terminal télécharge l'APK, vérifie l'empreinte, l'installe et le désigne propriétaire de l'appareil ; suite au §3.2 point 3.

Ce parcours n'a pas été exercé sur l'assistant réel du D3 Pro (EV-15, voir le début du paragraphe).

3.4 Premier démarrage, import du lot et identifiant de l'appareil

1. Écran « Mise en service de la caisse » : brancher la clé USB, toucher « Choisir le dossier » et choisir le dossier du lot.
2. L'application copie le lot, **vérifie la signature et chaque empreinte**, contrôle `pos.json`, applique `seed.sql` (base neuve seulement) et démarre le moteur : « Configuration importée. La caisse démarre. ». Sans lot valide, **le moteur ne démarre pas** : il n'y a aucune identité de test implicite.
3. Refus possibles : signature absente ou invalide, ou fichier manquant, modifié ou non prévu (le lot ne correspond pas à son manifeste : le régénérer et le signer) ; « La configuration est invalide : … » (corriger `pos.json`, signer, réimporter) ; « Appareil non préparé » (l'application n'est pas propriétaire : §3.2) ; « Numéro de série illisible » (moteur fermé, `PLATEFORME_ANDROID_FR.md` §4) ; « Configuration d'un autre appareil » (lot préparé pour un autre matériel : nouvel import signé sur ce terminal).
4. **Identifiant de l'appareil.** Il n'est pas saisi : l'application lit le numéro de série matériel (FAQ producteurs Q505 b) à chaque ouverture du moteur et le fait imprimer sur les tickets. Le relever dans l'écran « À propos » et le consigner au formulaire d'installation avec la version et le SHA-1 affichés (`PLATEFORME_ANDROID_FR.md` §4). Format réel : À effectuer (EV-15 : numéro de série réel). Valeur de la machine de test : `EMULATOR37X2X12X0` (profil émulé).
5. L'application est désormais l'écran d'accueil verrouillé du terminal (Lock Task, `PLATEFORME_ANDROID_FR.md` §3.2).

3.5 Services Google : aucun compte, masquage de la boutique et des applications

Le terminal porte les services Google, la caisse n'en utilise aucun.

- **Jamais de compte Google**, ni au banc ni chez le client : une fois propriétaire, l'application interdit d'en ajouter (`no_modify_accounts`). Sur un appareil qui en porte déjà un : réinitialisation d'un terminal neuf ou retiré du service, puis provisionnement au banc (§3.2).
- La boutique Play et les applications Google destinées à l'utilisateur sont **masquées par le propriétaire de l'appareil**, et toute installation est bloquée sauf par l'outil de mise à jour déclarée. Le code le fait au démarrage de l'application (liste des paquets au `PLATEFORME_ANDROID_FR.md` §3.7) ; le mode kiosque suspend en plus les autres écrans d'accueil et les Réglages. Mesuré sur le profil avec services Google : onze paquets Google présents et masqués, restrictions d'installation posées (EV-13, `evidence/gms/EV13_2026-10-15.md`). Tout message de Play Protect est noté tel quel au rapport d'essai.

4. Configuration initiale

4.1 Accès maintenance

Sans objet sur Android : ni SSH, ni console, ni compte de maintenance, ni assistance à distance, et adb est coupé en production. L'assistance se fait sur place, par les écrans de la caisse sous PIN de Gérant, un nouveau lot signé (§3.4) et l'export des données (`SECURITE_BD_FR.md` §8).

4.2 Fichier de configuration `pos.json`

Le fichier est celui du moteur, identique à celui de Linux. Il se place **à la racine du lot** (§2.1) et se lit à chaque démarrage ; ses chemins de fichiers sont relatifs au dossier du lot. Pour corriger une valeur, on refait un lot

signé et on le réimporte.

```
{
  "pos_id": "CFOD0061234567",
  "terminal_id": "POS1",
  "device_id": "pos-json-device",
  "restaurant_name": "Restaurant Exemple",
  "restaurant_legal_name": "Exemple Horeca SRL",
  "restaurant_address": "Rue de l'Exemple 1, 4000 Liège",
  "restaurant_vat": "BE0499999960",
  "restaurant_est_no": "2123456791",
  "fdm_url": "https://192.168.1.20/graphql/",
  "fdm_token_file": "secrets/fdm.token",
  "fdm_ca_file": "fdm/ca.crt",
  "fdm_client_cert_file": "fdm/client.crt",
  "fdm_client_key_file": "fdm/client.key",
  "printer_address": "192.168.1.30:9100",
  "kitchen_printer_address": "192.168.1.31:9100",
  "training_mode": false,
  "day_end_hour": 5,
  "table_count": 20,
  "language": "fr"
}
```

Champ	Description	Source
`pos_id`	14 caractères, lettres majuscules ou chiffres (commence par `C`). Attribué par le SPF Finances à la certification.	SPF Finances
`terminal_id`	ID du terminal physique, en général `POS1`, `POS2`…	Distributeur
`device_id`	**Requis par le format** (1 à 600 caractères, toute valeur non vide, par exemple `pos-json-device`) : sans lui l'import est refusé (« La configuration est invalide »). **Sans effet sur Android** : le moteur le remplace, à chaque ouverture, par le numéro de série matériel du terminal (§3.4).	Automatique
`restaurant_name`, `restaurant_address`	Nom commercial et adresse complète, imprimés sur le ticket	Client
`restaurant_legal_name`	Facultatif. Dénomination légale imprimée en bas du ticket avec l'adresse et la TVA ; absent : le nom commercial.	Client
`restaurant_vat`	BE suivi de 10 chiffres commençant par 0 ou 1, valide modulo 97	Client
`restaurant_est_no`	Unité d'établissement : 10 chiffres commençant par 2 à 8, valide modulo 97	BCE / client

Champ	Description	Source
`fdm_url`	URL GraphQL du FDM sur le LAN. Module2, configuration retenue : `https://<IP du FDM>/graphql/` (443, mTLS, §7.3) ; `http://<IP du FDM>/graphql/` (80) reste possible.	Configuration FDM
`fdm_token_file`	Fichier du lot contenant le seul jeton Bearer du FDM (page de paramètres du FDM ou portail module2) ; jamais le jeton dans `pos.json`.	Configuration FDM
`fdm_ca_file`	Requis en `https://` : autorité du FDM (PEM, `ca.crt` de module2).	Configuration FDM
`fdm_client_cert_file`, `fdm_client_key_file`	Ensemble ou pas du tout : `client.crt` et `client.key` de module2 pour un FDM en mTLS.	Configuration FDM
`printer_address`	IP:port de l'imprimante du LAN (9100 par défaut)	Configuration imprimante
`kitchen_printer_address`	Facultatif. Imprimante du bar ou de la cuisine (« BON DE COMMANDE » sans prix) ; absent : le bon sort sur `printer_address`.	Configuration imprimante
`training_mode`	`false` en production, **à ne jamais laisser `true`**. Valeur de départ : un Gérant peut changer le mode dans « Config » (§4.7).	Distributeur
`hubrise_enabled`, `hubrise`	Facultatifs. Commandes en ligne (`COMMANDES_EN_LIGNE_FR.md`) : `hubrise_enabled` à `true` et bloc `hubrise` (`client_id`, `client_secret_file` relatif au lot). Connexion ensuite par un Gérant : « Config », panneau « HubRise », code QR puis code d'autorisation.	Client
`table_count`	Nombre de tables du plan de salle, à partir de 1 ; défaut `20`.	Client
`day_end_hour`	Heure belge (0 à 12) de fin de journée commerciale ; défaut `5`. Une journée restée ouverte au-delà est clôturée par un rapport Z automatique (FAQ producteurs Q501).	Client
`language`	Langue par défaut de l'interface : `fr`, `nl`, `de`, `en`, `zh`. Les mentions fiscales imprimées suivent cette clé, restreinte à fr/nl/de (toute autre valeur : FR).	Client
`backup_dir`	**À ne pas renseigner sur Android** : les sauvegardes vont dans le stockage privé de l'application (§8).	-

Langue du ticket. `language` suit la **région linguistique de l'établissement** (manuel de certification §VI), ni le siège du distributeur ni la langue d'interface souhaitée : région française `fr`, néerlandaise `nl`, allemande `de`, Bruxelles-Capitale `fr` ou `nl` (`GARANTIE_MAJ_LANGUES_FR.md` §3.2).

Ticket numérique par e-mail (facultatif). Sans bloc `digital_ticket`, la caisse n'émet que du papier. Pour proposer l'e-mail (manuel de certification §III.H, `TICKET_NUMERIQUE_FR.md`), ajouter à `pos.json` le bloc `digital_ticket` : `project_id` (projet Scaleway Transactional Email), `from_email` (domaine vérifié), `from_name`, `api_key_file` (fichier du lot contenant la seule clé secrète), `allow_digital_only` (`true` permet « E-mail » seul, défaut `false`) et `region` (défaut `fr-par`). Seul trafic ajouté : HTTPS vers `api.scaleway.com`. Clé absente ou service muet : la caisse reste en papier et le consigne.

Export nocturne (facultatif). Sans section `export`, la caisse n'envoie rien hors du site. Pour envoyer, après chaque rapport Z réel, une sauvegarde chiffrée et une copie statistique au point de réception de l'installation (`EXPORT_NOCTURNE_FR.md`) :

```
"export": {
  "url": "https://<hôte de réception de l'installation>/<chemin>",
  "token_file": "secrets/export.token",
  "owner_recipient": "<clé publique age du propriétaire, age1...>",
  "producer_recipient": "<clé publique age du producteur, facultatif>",
  "keep_days": 14
}
```

`url` : HTTPS, sans identifiant ni paramètre. `token_file` : fichier du lot contenant le seul jeton. `owner_recipient` : clé **publique** age du propriétaire (62 caractères, `age1...`), obligatoire. `producer_recipient` : facultatif. `keep_days` : de 1 à 365, défaut 14 (copies locales dans `files/engine/exports`, jamais sur une carte SD). Le propriétaire génère sa paire sur son poste (`age-keygen -o cle-proprietaire.txt`) et garde la clé **privée** hors ligne : sans elle les sauvegardes envoyées ne se relisent plus, et rien ne permet de la retrouver. Jamais de clé privée dans le lot, sur la caisse ou sur le serveur de réception. L'état de l'export se lit dans « Export nocturne » (`MANUEL_UTILISATEUR_FR.md`).

Terminal de paiement par carte relié (facultatif). Sans section `payment_terminal`, rien ne change : la carte est payée sur un terminal de paiement autonome, que l'opérateur saisit à la caisse (méthode `MANUAL`, type `CARD_DEBIT`). Avec cette section, la caisse envoie le montant au terminal et enregistre sa réponse (ligne de paiement `AUTOMATIC`, `MANUEL_UTILISATEUR_FR.md` §7.2 ; une carte de chèques-repas Edenred, Pluxee, Monizze ou Payflip, reconnue à la marque que rend le terminal, est signée `CHEQUE_MEAL`). Trois protocoles sont pris en charge, chacun d'après une spécification publique ; `kind` choisit l'un d'eux :

`kind`	Protocole	Chemin
`worldline_terminal_api`	Worldline Terminal API (nexo Retailer, JSON sur HTTPS)	La caisse parle au nuage Worldline, qui joint le terminal (terminal Android Worldline avec l'application "Terminal API")
`ccv_payment_api`	CCV Payment API, méthode `terminal` (JSON sur HTTPS)	La caisse parle au nuage CCV, qui joint le terminal
`ccv_opi`	OPI (Open Payment Initiative, XML sur TCP)	Directement sur le LAN du restaurant, sans nuage

Le terminal Worldline C-TEP (gamme Yomani / Yoximo) n'est pas pris en charge : son protocole est fermé (bibliothèque et certification Worldline).

```
"payment_terminal": {
  "kind": "worldline_terminal_api",
  "merchant_id": "<UMID>",
  "terminal_id": "<UTID>",
  "integrator_id": "<identifiant d'intégrateur>",
```

```
  "api_key_file": "secrets/terminal.key"
}
```

Champ	Description
`kind`	`worldline_terminal_api`, `ccv_payment_api` ou `ccv_opi`. Obligatoire.
`base_url`	Types nuage seulement. Absent : l'adresse de production du fournisseur. À renseigner pour l'environnement de test du fournisseur ou pour un relais (voir plus bas).
`api_key_file`	Types nuage seulement. Fichier du lot contenant la seule clé d'API, jamais la clé elle-même dans `pos.json` (chemin relatif au dossier du lot, comme `fdm_token_file`). Absent seulement quand `base_url` est un relais qui authentifie la caisse lui-même.
`merchant_id`, `terminal_id`, `integrator_id`	Worldline, tous trois obligatoires : identifiant du commerçant (UMID), du terminal (UTID) et de l'intégrateur que Worldline délivre au producteur.
`terminal_id`, `management_system_id`	CCV Payment API, tous deux obligatoires : identifiant du terminal attribué par CCV et identifiant du système de gestion du terminal.
`access_protocol`	CCV Payment API, facultatif : protocole d'accès du terminal, `OPI_NL` si absent.
`address`	OPI, obligatoire : `hôte:port` du terminal sur le LAN (canal 0, en général le port 4100).
`device_port`	OPI, facultatif : port d'écoute de la caisse pour le canal 1 du terminal (affichage et impression), 4102 si absent.
`terminal_id`	OPI, facultatif : identifiant de poste envoyé au terminal.
`timeout_seconds`	Facultatif, 30 à 600, 180 si absent : attente la plus longue de la réponse du terminal pour un paiement. Passé ce délai, le résultat est redemandé (statut inconnu, voir `MANUEL_UTILISATEUR_FR.md` §7.2).

Règle CCV : la clé d'API CCV ne doit pas se trouver sur un appareil du restaurant. Pour `ccv_payment_api`, renseigner alors `base_url` avec l'adresse d'un relais du distributeur, qui détient la clé et authentifie la caisse, et ne pas mettre `api_key_file`. Pour Worldline, la clé est un fichier de secret du lot, comme le jeton du FDM : on la dépose dans le dossier des secrets du lot (fichier `terminal.key`) **sur le poste du producteur**, puis on signe le lot (§2.1). À l'import, l'application signale un champ manquant ou un fichier vide et refuse le lot ; le terminal de paiement lui-même n'est pas contacté à ce moment.

Le seul trafic sortant ajouté est HTTPS (443) vers le nuage du fournisseur (types nuage), ou une connexion TCP vers le terminal sur le LAN (`ccv_opi` : la caisse ouvre le canal 0 vers `address` et écoute sur `device_port` pour le canal 1 ; aucun autre port entrant). Sur Android, ce réseau est celui de la tablette : le Wi-Fi ou le réseau filaire du restaurant, le même que pour le FDM et l'imprimante (§4.3). Chaque fournisseur certifie lui-même l'intégration avant tout usage réel et délivre les identifiants nécessaires : la configuration d'un établissement n'est livrée qu'avec ces éléments. Les paiements par carte ne sont jamais arrondis, pour toute installation. La caisse montre si le terminal relié répond (voyant « Terminal » de l'en-tête, ligne « Terminal de paiement » du panneau « Démarrage », type du terminal dans Configuration, « Informations »).

Vérification. À l'import, l'application applique à `pos.json` les mêmes contrôles que `pili-pos --check-config` sous Linux (format du `pos_id`, TVA et unité d'établissement modulo 97, champs vides, URL du FDM, langue, blocs `digital_ticket`, `export` et `payment_terminal`) ; un champ refusé bloque l'import et le mes-

sage nomme la cause. Le FDM refusant tout événement à l'identification mal formée (description détaillée POS v2.2 §2.2.2), contrôler le lot sur le poste avant de partir.

4.3 Configuration réseau LAN

Le terminal, le FDM et l'imprimante sont sur le **même LAN privé**. Plan type, subnet `192.168.1.0/24`, réservations DHCP sur le routeur : terminal `192.168.1.10`, FDM `192.168.1.20` (HTTPS 443, §7.3), imprimante `192.168.1.30` (TCP 9100), box `192.168.1.1` (`SCHEMA_ASSEMBLAGE_FR.md` §4). Le terminal n'ouvre aucun port en écoute. Il se relie en Wi-Fi ou par câble si son modèle en offre un.

Comme construit en 1.0.0, l'application n'a aucun écran de réseau : le mode kiosque suspend les Réglages d'Android (`PLATEFORME_ANDROID_FR.md` §3.2) et le Gérant ne peut donc pas saisir un réseau sur place. Le Wi-Fi du site (SSID et mot de passe) se prépare **au banc, avant l'import du lot** (§3.2 point 5), pendant que le débogage USB est encore actif : il est coupé dès qu'un enregistrement de provisionnement existe (`PLATEFORME_ANDROID_FR.md` §3.3). Sur un site sans Wi-Fi, utiliser le câble si le modèle en offre un. Changer de réseau ou de mot de passe après la livraison n'est pas possible sur place en 1.0.0, et ne se règle jamais par une réinitialisation ni une désinstallation chez le client (§9) : il faut une version déclarée plus récente (§6). L'écran de réseau protégé par le code d'un Gérant, qui rendrait ce changement possible sur place, est la **cible** et n'est pas construit : À effectuer (EV-10 : parcours sur les deux profils du D3 Pro). Wi-Fi et adresse MAC du vrai terminal : À effectuer (EV-15).

4.4 Configuration utilisateurs

Il n'y a pas de base à ouvrir à la main. Les premiers comptes viennent de deux voies :

1. **`seed.sql` du lot**, appliqué une seule fois, avant le premier événement fiscal. Pour un établissement, le technicien y place au moins le Gérant (mêmes instructions SQL que sous Linux) :

   ```
   INSERT INTO users (insz, name, role, active)
   VALUES ('<INSZ du Gérant, 11 chiffres>', '<Nom>', 'admin', 1);
   ```

   Rôles : `admin` (Gérant), `cashier` (Serveur), `waiter` (Apprenti). Machine de test du SPF : graine du jeu de test (`src-tauri/seed_spf_test.sql`, comptes du §7.1).

2. **« Config » d'un Gérant connecté** : ajout, désactivation et PIN des employés (`MANUEL_UTILISATEUR_FR.md`), pour les comptes suivants.

Aucun PIN dans la graine. Le premier Gérant choisit le sien à sa première connexion (sa tuile ouvre « Choisissez votre PIN ») : le technicien reste présent jusqu'à ce choix. Un deuxième Gérant reçoit son PIN du premier, dans « Config », confirmé par le PIN de ce dernier ; le Serveur et l'Apprenti fixent le leur en touchant leur nom. Ne pas créer l'INSZ `00000000029` : identité système, qui ne se connecte jamais. Le catalogue est livré par la graine ou par HubRise (`MANUEL_PROGRAMMATION_FR.md`).

4.4 bis Lecteur de badges (option)

Facultatif et **désactivé par défaut** : sans lecteur, rien à faire. Pour pointer et se connecter avec un badge (`MANUEL_UTILISATEUR_FR.md` §12.13) :

- **Lecteur USB HID « clavier ».** Brancher un lecteur de cartes NFC USB qui se comporte comme un clavier : il « tape » le numéro de la carte (identifiant de série, en hexadécimal si le lecteur le permet, dans l'ordre des octets du lecteur) puis la touche **Entrée**. Aucun pilote ; la disposition du clavier (AZERTY compris) n'a pas d'importance, car la caisse lit les touches physiques. Le lecteur doit envoyer Entrée à la fin (sinon le configurer ainsi) ; préfixes et suffixes autres que Entrée ne sont pas pris en charge.
- Voir plus bas pour le NFC intégré.
- Un seul type de lecteur par caisse : deux lecteurs peuvent écrire la même carte différemment (hexadécimal ou décimal, ordre des octets). Utiliser le même lecteur pour tous les badges, sans quoi les badges sont à lier de nouveau.

- Les réglages (Désactivé, Lecteur USB, NFC intégré, connexion par badge, délais du lecteur) se font ensuite dans « Config », panneau « Lecteur de badges », par le Gérant ; la clé qui protège les badges est créée sur l'appareil à la première utilisation, reste sur l'appareil et n'est ni sauvegardée ni exportée (`SECURITE_BD_FR.md` §2.5).
- **Android uniquement : clavier virtuel.** Un lecteur USB est un clavier physique pour Android, qui peut alors masquer le clavier à l'écran. Dans les réglages Android (accès maintenance, §4.1 : Système, Langues et saisie, Clavier physique), laisser activée l'option qui affiche le clavier virtuel tant qu'un clavier physique est connecté (libellé « Afficher le clavier virtuel » ou « Utiliser le clavier à l'écran » selon la version). Sinon les champs de texte du panneau « Employés » ne seraient plus saisissables.
- **Android : NFC intégré.** L'unité principale du Sunmi D3 Pro n'a pas de NFC : choisir « Lecteur USB ». Le réglage « NFC intégré » n'apparaît que sur un appareil qui déclare un lecteur NFC (`FEATURE_NFC`) ; le NFC de l'écran client optionnel du D3 Pro n'est utilisable que s'il est présenté ainsi (à confirmer sur place avec une carte de test, voir `PLATEFORME_ANDROID_FR.md`). L'application s'installe aussi sur un appareil sans NFC (la fonction est facultative).
- **Test après l'installation.** Dans « Config », « Lecteur de badges » : choisir « Lecteur USB », puis « Lier un badge » sur un employé de test (pas le Gérant connecté), présenter la carte, confirmer avec le PIN du Gérant, puis présenter de nouveau la carte depuis l'écran de commande : « Arrivée · nom · heure » (ou « Départ »), l'événement S est signé par le FDM. Retirer ensuite le badge de test et remettre le réglage voulu par l'établissement.

4.5 Synchronisation horaire

Pas de service à vérifier : le propriétaire de l'appareil verrouille la date et l'heure, active l'heure automatique par le réseau et fixe le fuseau sur Europe/Brussels (`PLATEFORME_ANDROID_FR.md` §3.4). À la mise en service, comparer l'horloge à une référence : elle **doit être exacte à la seconde près** (`posDateTime` cohérent à la signature du FDM). Le terminal doit joindre Internet au moins une fois pour se régler.

4.6 Test du flux complet

Avant de quitter le site :

1. Éteindre et rallumer le terminal : l'application revient seule au premier plan, verrouillée, en quelques secondes (émulateur : EV-10 ; vrai terminal : À effectuer (EV-15 : redémarrage sur le D3 Pro réel)).
2. À l'écran de connexion, contrôle du FDM « connecté » et version pili-pos-android/1.0.0 ; se connecter ; le pointage S doit signer.
3. Ouvrir une table, ajouter un article, « Envoyer en cuisine » : le bon s'imprime sur l'imprimante de la cuisine, ou sur celle des tickets si `kitchen_printer_address` est absent.
4. Encaisser en espèces (ticket avec QR code, tiroir ouvert), puis « Rapports », onglet « Journée », « Rapport X ».
5. Sur la machine de test, exporter vers le SPF (§4.7) et ouvrir `pos.db` avec `sqlite3 -readonly` : au moins un S, un N et, si envoyé en cuisine, un P.

```
SELECT id, event_type, sequence_number, short_signature
FROM fiscal_events ORDER BY id DESC LIMIT 5;
```

4.7 Paramètres de l'installation, journaux, réseau et comptes

Le technicien seul modifie l'identité et le réseau (`pos_id`, `restaurant_*`, `fdm_*`, imprimantes, `language`, `day_end_hour`, `table_count`, `digital_ticket`, `hubrise`, `export`), par un nouveau lot signé. `training_mode` et `hubrise_enabled` ont leur valeur de départ dans le lot, puis le Gérant les change dans « Config » (panneaux « Mode Formation » et « Intégrations » ; le premier réglage vaut jusqu'au redémarrage, chaque changement du second est journalisé). La langue d'affichage (panneau « Langue ») n'a pas d'effet sur les mentions imprimées. Les employés et leurs PIN sont à la charge du technicien (graine) puis du Gérant (§4.4).

Journaux. Le moteur écrit un fichier par jour dans le dossier des journaux de l'application et la plateforme tient son journal d'audit (PLATEFORME_ANDROID_FR.md §3.8) ; ils ne se lisent que par l'export vers le SPF. fiscal_events est en ajout seul (déclencheurs SQL). Une sauvegarde de pos.db est écrite après chaque rapport Z et avant chaque mise à jour (§8).

Réseau. Sortant seulement : FDM (HTTPS 443 ou HTTP 80) et imprimante (TCP 9100) sur le LAN, api.scaleway.com si digital_ticket, manager.hubrise.com et api.hubrise.com si HubRise, le point de réception de l'export nocturne si export, et l'artefact public de mise à jour quand un Gérant la lance. Aucun port en entrée (PLATEFORME_ANDROID_FR.md §6).

Comptes. Aucun compte système ni de contrôle : les employés sont les lignes de users (INSZ et PIN). Pas d'équivalent du compte spf de l'image d'admission Linux : le SPF accède aux données par l'export du Gérant, « Export pour le SPF » (SECURITE_BD_FR.md §8.2), vers une clé USB ou une carte SD en exFAT.

5. Vérification de l'intégrité de l'APK

Le SHA-1 de l'APK signé certifié est consigné dans le formulaire SPF Finances. Sur la caisse il se lit dans l'écran « À propos » (SHA-1 et SHA-256 de base.apk, SHA-1 de libpos_ffi.so, SHA-256 du certificat de signature), jamais par un outil branché : adb est coupé en production. Les valeurs doivent être égales à EV-01, EV-02 et EV-03 (GARANTIE_MAJ_LANGUES_FR.md §1.2), sinon **NE PAS METTRE EN PRODUCTION**.

6. Mise à jour (procédure post-cert)

La procédure officielle du SPF Finances (Manuel_Intro_ToRead_FR.md §C) s'applique : déclaration de la modification (changelog signé, affirmation qu'aucune autre modification n'a été apportée, nouveau SHA-1), délai de 15 jours, déploiement après accord (GARANTIE_MAJ_LANGUES_FR.md §2.1).

Le nouvel APK n'est **jamais** installé à distance ni par adb : le Gérant, sur place, ouvre « Config », « Mise à jour », puis « Vérifier en ligne » (artefact public declared-android) ou « Depuis une clé USB » (les trois fichiers de la déclaration remis par le technicien). L'outil refuse si un rapport Z est dû, si une commande est ouverte, ou si signature ou empreintes ne correspondent pas ; il sauvegarde pos.db, installe, redémarre l'application seul et consigne le SHA-1 installé. Ensuite : lire le SHA-1 dans « À propos » (§5), faire le test abrégé du §4.6 et vérifier que l'historique fiscal est intact. **Pas de retour arrière** : une version défectueuse se corrige par une nouvelle version déclarée.

7. Annexes

7.1 Liste des numéros INSZ de test

Comptes de la graine du jeu de test du SPF (src-tauri/seed_spf_test.sql), à l'usage exclusif de la phase de test pré-soumission :

Rôle	Nom	INSZ	PIN
Serveur	John (barman)	75061189702	4444
Serveur	Elisa (chef de salle)	84022899837	2222
Serveur	Sammy (sommelier)	80113078968	5555
Gérant	Gerant Test	94060242576	1111
Gérant	Technicien SPF	00000000097	0097
Apprenti	Apprenti Test	99012345667	3333
Robot (identité système, sans connexion)	Robot / Kiosk	00000000029	-

7.2 Numéros pos_id de test

CPOS0031234567 : posId de la configuration de test du SPF Finances (UC_TestSetup_20240806), autorisé sur le simulateur module2. Terminal de la machine de test : POS-2-ANDROID.

7.3 FDM

- **FDM module2** (PRACSYS, certifié SCE 2.0), référence retenue : GraphQL sur le LAN en HTTPS (443, certificat client mTLS) ; HTTP (80) reste possible. Les trois fichiers de module2 (`ca.crt`, `client.crt`, `client.key`, publiés sur manual.module2.be) vont dans le lot, désignés par `fdm_ca_file`, `fdm_client_cert_file` et `fdm_client_key_file`. La caisse vérifie que le certificat du FDM est signé par cette autorité et valide (il ne porte pas de nom d'hôte). Chaque FDM exige son jeton Bearer (`fdm_token_file`) et le posId doit y être autorisé.
- **Simulateur en ligne module2** : `https://fdm.module2.be/graphql/`, mêmes certificats et jeton ; FDM de la machine de test à l'Intake (`SCHEMA_ASSEMBLAGE_FR.md` §7). Essai en direct sur accord du producteur : À effectuer (EV-14 : essai en direct contre le simulateur module2).

7.4 Contact distributeur

Bruno Moyse / nuagemagique.dev, cloud@nuagemagique.dev, Bruxelles.

7.5 Dépannage : réponse du FDM en attente

Symptôme : toute vente, tout rapport ou tout pointage est refusé avec « La caisse attend la réponse de la boîte noire pour l'opération précédente » (code `FDM_ANSWER_PENDING`). La dernière requête est partie sans réponse en retour ; elle est conservée telle quelle et renvoyée à l'identique toutes les 60 secondes (un FDM qui l'avait signée répond `DUPLICATE_REQUEST` et l'événement est archivé une seule fois). Pas d'abandon automatique. Vérifier câble ou Wi-Fi, alimentation et voyant du FDM, attendre une minute, puis contrôler son état (tampon plein, FDM verrouillé) et contacter le distributeur si le blocage dure. **Ne jamais refaire la vente à la main** : la requête en attente est intouchable. Si le FDM la refuse, elle passe seule à « refusée », son numéro est libre et la caisse repart.

8. Stockage : espace interne et carte SD

Chiffres de `PLATEFORME_ANDROID_FR.md` §5, mesurés sur l'émulateur.

- **La base n'est jamais sur la carte SD.** `pos.db` est dans le stockage interne privé de l'application, avec ses sauvegardes. La carte est un volume amovible atteint par le sélecteur d'Android : cible de l'export vers le SPF et source d'une mise à jour. Ne pas l'adopter comme stockage interne, ne pas déplacer l'application.
- **Besoin interne.** Après chaque rapport Z, la caisse écrit une copie de `pos.db` et **conserve les 7 plus récentes**, en supprimant les plus anciennes ; le journal fiscal de `pos.db` n'est jamais supprimé (commit du moteur : À effectuer (EV-19 : commit du changement du moteur)). Elle occupe environ 8 fois la base : environ 4 Go après un an à 100 ventes par jour, environ 13 Go à 300. « À propos » affiche l'espace libre et avertit sous 2 Gio. Espace libre du vrai D3 Pro : À effectuer (EV-15).
- **Carte SD recommandée.** microSDXC de qualité, haute endurance (A1 ou U3), en **exFAT** (le FAT32 refuse les fichiers de 4 Gio et plus) : **64 Go** la première année à environ 100 ventes par jour, **128 Go** à environ 300 ventes par jour ou dès la deuxième année. Un export pèse environ 8 fois la base (la base et les 7 sauvegardes). L'application n'efface jamais un dossier de la carte : le Gérant supprime les anciens exports.

9. Ce que l'installateur ne fait jamais

- **Réinitialiser aux paramètres d'usine un terminal en service, ou désinstaller l'application** : la base fiscale et ses 7 sauvegardes sont dans le stockage privé de l'application et disparaissent avec elle. La restriction `no_factory_reset` bloque la réinitialisation depuis les Réglages et `no_uninstall_apps` la désinstallation ; le menu de récupération du fabricant n'est pas établi (EV-15, `PLATEFORME_ANDROID_FR.md` §3.3). L'interdiction reste absolue : ne pas chercher à contourner ces restrictions. Avant de retirer un terminal, exporter les données (§4.7) et conserver la clé ou la carte.
- **Se connecter à un compte Google**, installer une application ou utiliser la boutique Play (§3.5), ou laisser la clé du lot chez le client.

- Installer un APK autrement que par l'outil de mise à jour (§6), ou une version de développement (`–dev`).
- Laisser `training_mode` à `true`, ou renseigner `backup_dir` (§4.2).
- Remplacer un terminal défaillant sans nouveau lot signé : le moteur reste fermé tant que le numéro de série ne correspond pas à l'enregistrement (§3.4). La reprise des données de l'ancien terminal n'est pas décrite ici : À effectuer (EV-15 : procédure de remplacement sur un vrai D3 Pro).